



Generatori

Python

Un generatore in Python è un costrutto che permette di produrre una sequenza di valori in modo lazy (pigro), cioè uno alla volta, solo quando richiesti, invece di calcolarli e memorizzarli tutti in memoria in anticipo.

Si realizza tramite funzioni che utilizzano la parola chiave `yield` al posto di `return`: ogni volta che il generatore viene iterato, l'esecuzione della funzione riprende dal punto in cui era stata sospesa, mantenendo il proprio stato interno.

Questo comportamento lo distingue dalle funzioni tradizionali e dalle strutture dati come liste o tuple, rendendolo particolarmente efficiente in termini di memoria e adatto a sequenze potenzialmente infinite o molto grandi.



I generatori sono utilizzati in diversi casi pratici: per gestire grandi dataset senza caricarli completamente in memoria (ad esempio lettura riga per riga di file molto grandi), per creare stream di dati come sequenze numeriche o eventi in tempo reale, per implementare pipeline di elaborazione dove i dati passano attraverso più trasformazioni successive, e per definire iterazioni personalizzate più leggibili ed efficienti rispetto alla costruzione manuale di iteratori.

Sono anche alla base delle generator expressions (simili alle list comprehension ma lazy), e risultano particolarmente utili quando si lavora con input continui, API o simulazioni.



Lettura di file molto grandi

Quando si lavora con file enormi (log, dataset), caricare tutto in memoria con `readlines()` è inefficiente.

Un generatore consente di leggere una riga alla volta.

```
1. def leggi_file(nome_file):  
2.   with open(nome_file) as f:  
3.     for riga in f:  
4.       yield riga
```

Uso: analisi di log di sistema o dataset giganti senza saturare la RAM.



Generazione di sequenze (anche infinite)

I generatori sono ideali per sequenze potenzialmente infinite, perché producono valori solo quando richiesti.

```
1. def numeri_infiniti():  
2.   n = 0  
3.   while True:  
4.       yield n  
5.       n += 1
```

Uso: simulazioni, contatori, o algoritmi che non hanno un limite predefinito.



Pipeline di elaborazione dati

I generatori possono essere concatenati per creare pipeline efficienti.

```
1. def filtra_pari(neri):  
2.     for n in neri:  
3.         if n % 2 == 0:  
4.             yield n  
5.  
6. def raddoppia(neri):  
7.     for n in neri:  
8.         yield n * 2  
9.  
10. neri = range(10)  
11. pipeline = raddoppia(filtra_pari(neri))  
12. print(list(pipeline)) # [0, 4, 8, 12, 16]
```



Uso: elaborazione dati in streaming (es. ETL, trasformazioni progressive).

Gestione di stream o API

Quando i dati arrivano nel tempo (es. da API o sensori), i generatori permettono di consumarli progressivamente.

```
1. def stream_dati():  
2.   import time  
3.   for i in range(5):  
4.     time.sleep(1)  
5.     yield f"dato {i}"
```

Uso: dati in tempo reale, sistemi IoT, aggiornamenti live.



Iterazioni controllate in simulazioni o giochi

Nei contesti come simulazioni o game loop, i generatori possono gestire stati progressivi.

```
1. def animazione():  
2.     frame = 0  
3.     while frame < 3:  
4.         yield f"frame {frame}"  
5.         frame += 1
```

Uso: aggiornamento step-by-step di stati (utile anche nel game dev).



Generator expressions (versione compatta)

Forma breve e leggibile per creare generatori.

```
1. quadrati = (x**2 for x in range(5))  
2.  
3. for q in quadrati:  
4.   print(q)
```

Uso: alternativa più efficiente alle liste quando non serve memorizzare tutto.



- 1. `leggi_file()` → Lazy loading
Non carica tutto il file: restituisce una riga alla volta, risolve il problema della memoria con file grandi.

- 2. `filtra_errori()` → Selezione dati
Riceve un generatore e produce solo le righe che contengono "ERROR".
Nessuna lista intermedia → efficienza.

- 3. `estrai_messaggio()` → Trasformazione
Trasforma ogni riga in un dato più utile.
Pipeline modulare (stile ETL).

- 4. `processa_log()` → Composizione
Concatena i generatori: ogni dato passa attraverso gli step uno alla volta.
Streaming completo.

- 5. `for` finale → Consumo
I dati vengono effettivamente generati solo qui. Esecuzione "on demand".

```
1. def leggi_file(nome_file):
2.     """Generatore: legge il file riga per riga"""
3.     with open(nome_file) as f:
4.         for riga in f:
5.             yield riga
6.
7. def filtra_errori(righe):
8.     """Generatore: filtra solo le righe con ERROR"""
9.     for r in righe:
10.        if "ERROR" in r:
11.            yield r
12.
13. def estrai_messaggio(righe):
14.     """Generatore: estrae solo il messaggio utile"""
15.     for r in righe:
16.         yield r.split(" - ")[-1].strip()
17.
18. def processa_log(file):
19.     """Pipeline completa"""
20.     return estrai_messaggio(
21.         filtra_errori(
22.             leggi_file(file)
23.         )
24.     )
25.
26. # Uso
27. for messaggio in processa_log("log.txt"):
28.     print(messaggio)
```



Buon Davante a tutti

